Лабораторная работа 1

Тема: обработка признаков.

Используем датасет Adult Income: в нем есть числовые признаки, категориальные признаки и пропуски, записанные как `?`. Если OpenML недоступен, код создаст небольшой запасной набор с той же структурой.

In [1]:
```
import importlib.util
import subprocess
import sys


def ensure(package, import_name=None):
    import_name = import_name or package
    if importlib.util.find_spec(import_name) is None:
        subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q',

for package, import_name in [
    ('numpy', 'numpy'),
    ('pandas', 'pandas'),
    ('matplotlib', 'matplotlib'),
    ('scikit-learn', 'sklearn'),
]:
    ensure(package, import_name)

import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.datasets import fetch_openml
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder

pd.set_option('display.max_columns', 30)
RANDOM_STATE = 42
```

In [2]:
```
try:
    adult = fetch_openml('adult', version=2, as_frame=True, parser='auto')
    df = adult.frame.copy().replace('?', np.nan)
    target_col = adult.target.name if adult.target is not None else 'class'
    df = df.sample(n=min(5000, len(df)), random_state=RANDOM_STATE).reset_ir
    source = 'Adult Income, OpenML'
except Exception as exc:
    rng = np.random.default_rng(RANDOM_STATE)
    n = 300
    df = pd.DataFrame({
        'age': rng.integers(18, 70, n).astype(float),
        'hours_per_week': rng.normal(40, 12, n).round(1),
        'capital_gain': rng.gamma(2, 600, n).round(0),
```

```
        'workclass': rng.choice(['Private', 'Self-emp', 'Government', None],
        'education': rng.choice(['Bachelors', 'HS-grad', 'Masters', 'Some-cc
        'marital_status': rng.choice(['Never-married', 'Married', 'Divorced'
        'income': rng.choice(['<=50K', '>50K'], n, p=[0.72, 0.28]),
    })
    for col in ['age', 'hours_per_week']:
        df.loc[rng.choice(n, size=18, replace=False), col] = np.nan
    target_col = 'income'
    source = f'fallback dataset because OpenML failed: {type(exc).__name__}'

print(source)
df.head()
```

```
Adult Income, OpenML
```

Out[2]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship
0	56	Private	33115	HS-grad	9	Divorced	Other-service	Unmarried
1	25	Private	112847	HS-grad	9	Married-civ-spouse	Transport-moving	Own-child
2	43	Private	170525	Bachelors	13	Divorced	Prof-specialty	Not-in-family
3	32	Private	186788	HS-grad	9	Married-civ-spouse	Transport-moving	Husband
4	39	Private	277886	Bachelors	13	Married-civ-spouse	Sales	Wife

In [3]:
```
print('Размер данных:', df.shape)
missing = df.isna().sum().sort_values(ascending=False)
missing[missing > 0]
```

```
Размер данных: (5000, 15)
```

Out[3]:
```
workclass         281
occupation        281
native-country     88
dtype: int64
```

Разделяем признаки по типам. Целевую переменную не обрабатываем как обычный входной признак.

In [4]:
```
X = df.drop(columns=[target_col])
y = df[target_col]

numeric_cols = X.select_dtypes(include=np.number).columns.tolist()
categorical_cols = X.select_dtypes(exclude=np.number).columns.tolist()
X[categorical_cols] = X[categorical_cols].astype('object')
```

```
print('Числовые признаки:', numeric_cols)
print('Категориальные признаки:', categorical_cols[:10], '...' if len(categ
```

```
Числовые признаки: ['age', 'fnlwgt', 'education-num', 'capital-gain', 'capit
al-loss', 'hours-per-week']
Категориальные признаки: ['workclass', 'education', 'marital-status', 'occup
ation', 'relationship', 'race', 'sex', 'native-country']
```

In [5]:
```
try:
    encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    encoder = OneHotEncoder(handle_unknown='ignore', sparse=False)

numeric_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', MinMaxScaler()),
])

categorical_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', encoder),
])

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_pipeline, numeric_cols),
    ('cat', categorical_pipeline, categorical_cols),
])

X_processed = preprocessor.fit_transform(X)
feature_names = preprocessor.get_feature_names_out()

print('Было признаков:', X.shape[1])
print('Стало признаков после кодирования:', X_processed.shape[1])
print('Осталось пропусков:', np.isnan(X_processed).sum())
```

```
Было признаков: 14
Стало признаков после кодирования: 103
Осталось пропусков: 0
```

In [6]:
```
processed_preview = pd.DataFrame(X_processed[:5], columns=feature_names)
processed_preview.iloc[:, :15]
```

Out[6]:

	num__age	num__fnlwgt	num__education-num	num__capital-gain	num__capital-loss	num__h per-
0	0.534247	0.009946	0.533333	0.000000	0.0	0.3
1	0.109589	0.065446	0.533333	0.000000	0.0	0.3
2	0.356164	0.105595	0.800000	0.143441	0.0	0.3
3	0.205479	0.116915	0.533333	0.000000	0.0	0.3
4	0.301370	0.180327	0.800000	0.000000	0.0	0.2

In [7]:
```
num_after = pd.DataFrame(
    preprocessor.named_transformers_['num'].transform(X[numeric_cols]),
```

```
    columns=numeric_cols,
)

pd.DataFrame({
    'min_after': num_after.min(),
    'max_after': num_after.max(),
    'mean_after': num_after.mean().round(3),
}).head(12)
```

Out[7]:

	min_after	max_after	mean_after
age	0.0	1.0	0.298
fnlwgt	0.0	1.0	0.119
education-num	0.0	1.0	0.608
capital-gain	0.0	1.0	0.012
capital-loss	0.0	1.0	0.020
hours-per-week	0.0	1.0	0.401

В результате пропуски заменены, категории закодированы one-hot представлением, а числовые признаки приведены к диапазону от 0 до 1.